

Programming Language Principles, Design, and Implementation

Lecture 2 - Functional Programming in Haskell

Sean Moss

University of Birmingham

Where are we?

- ▶ **Part 1: Principles of programming**
 - ▶ **Functional languages**
 - ▶ Operational semantics
 - ▶ Type systems
 - ▶ Abstract datatypes
- ▶ Part 2: Compilers

Today

Functional Programming in Haskell

- ▶ Variables
- ▶ Types
- ▶ Functions
- ▶ Datatypes
- ▶ Pattern-matching
- ▶ Recursion
- ▶ Higher-Order Functions
- ▶ Laziness

Further reading:

- ▶ “A Gentle Introduction to Haskell”
<https://www.haskell.org/tutorial/>
- ▶ Almost anything you want to know about Haskell can be found there: <https://www.haskell.org>
- ▶ UPenn CIS 194 lectures:
<https://www.seas.upenn.edu/~cis194/>
- ▶ Online Haskell programming: <http://code.world/haskell>

Recap: Variables and Numbers

Remember: a Haskell program must have a “main” function:

- ▶ use for example: `main = print "done"`
- ▶ not necessary when using `ghci`

We can declare variables:

```
1 someint = 7
```

- ▶ `someint` is not mutable: it cannot be changed
- ▶ `someint` is the name of the value 7

Definitions can refer to themselves:

```
1 someint = 7 + someint
```

Recap: Variables, Numbers and Types

We can specify the type of definitions:

```
1 someint :: Int
2 someint = 7
```

- ▶ **Int**: machine-sized integers
- ▶ **Integer**: arbitrary size

Other kinds of numbers – double-precision floating point numbers:

```
1 somedouble :: Double
2 somedouble = 7.3
```

Recap: Variables, Numbers and Types

We can specify the type of definitions:

```
1 someint :: Int
2 someint = 7
```

What about?

```
1 someotherint :: Int
2 someotherint = 7 + someotherint
```

Haskell accepts this code because

- ▶ 7 is an **Int**
- ▶ and someotherint is also declared to be an **Int**

If the expression ever returned a value, it would be an “actual” integer.

Recap: Functions

We can manipulate data by application functions to that data:

```
1 increment :: Int -> Int
2 increment x = x + 1
3
4 inc7 :: Int
5 inc7 = increment 7
```

- ▶ increment is a function from **Int** to **Int** as indicated by $\rightarrow$
- ▶ it takes an x and returns $x+1$
- ▶ we don't need parentheses to apply increment to 7

Use parentheses if arguments are more complex:

```
1 inc7m2 :: Int
2 inc7m2 = increment (7 * 2)
```

Datatypes, Pattern-Matching & Recursion

A simple data type:

```
1 data Fruit = Apple | Pear | Banana
```

- ▶ A new type
- ▶ Apple constructs (is) an element of type Fruit
- ▶ Pear constructs (is) an element of type Fruit
- ▶ Banana constructs (is) an element of type Fruit
- ▶ We could have used numbers: 0 for Apple, 1 for Pear, 2 for Banana
 - ▶ Not convenient (what about 3, 4, ...?)
 - ▶ Easy to mix them up
 - ▶ Less readable

Another simple data type:

```
1 data Color = Green | Yellow
```


Datatypes, Pattern-Matching & Recursion

A simple data type:

```
1 data Fruit = Apple | Pear | Banana
2 data Color = Green | Yellow
```

Given an expression e of a datatype, we can **pattern-match** on e , i.e., we can specify **patterns** along with functions to execute if e conforms to the corresponding pattern.

In particular functions can pattern-match on their arguments.

Example:

```
1 whatColor :: Fruit -> Color
2 whatColor Apple  = Green
3 whatColor Pear   = Green
4 whatColor Banana = Yellow
```

Type classes I

```
1 data Fruit = Apple | Pear | Banana
2 data Color = Green | Yellow deriving Show
```

- ▶ **Show** is a **Type Class** that provides a **show** operations.
- ▶ **show** is used by **print** to get string representations.
- ▶ A type class specifies some behaviour (a set of functions), which instances have to implement.
- ▶ The implementation can be customized depending on the type of data in the datatype.
- ▶ Haskell can automatically derive some of these implementations using **deriving**

Datatypes, Pattern-Matching & Recursion

Booleans:

```
1 data Bool = True | False
```

Warning: The above **Bool** type is defined in the standard library, so either use the one from the library (don't redefine it) or use something like `(data B = T | F)` to avoid confusion

Write a function that takes 2 fruits and returns **True** iff they have the same color

For example:

```
1 sameColor :: Fruit -> Fruit -> Bool
2 sameColor Apple Apple = True
3 sameColor Apple Pear  = True
4 sameColor Pear  Apple = True
5 sameColor Pear  Pear  = True
6 sameColor Banana Banana = True
7 sameColor _      _      = False
```

Type classes II

```
1 data Fruit = Apple | Pear | Banana
2 data Color = Green | Yellow deriving (Show, Eq)
```

- ▶ **Eq** is a **Type Class** that provides an equality `==` binary function
- ▶ Haskell can automatically prove that `Color` is a member of the **Eq** class
- ▶ Now we can use the comparison operator `==`

Using type classes:

```
1 sameColor :: Fruit -> Fruit -> Bool
2 sameColor a b = whatColor a == whatColor b
```

Datatypes, Pattern-Matching & Recursion

Maybe/Option:

```
data Maybe a = Just a | Nothing
```

- ▶ To specify whether something is there or not
- ▶ Useful to define partial functions – return **Nothing** in the undefined cases
- ▶ **Maybe** is parameterized by the type variable *a*
- ▶ It is a **parameterized type constructor**: given a type it creates a new type
- ▶ E.g., given **Int** it creates **Maybe Int**
- ▶ E.g., given **Bool** it creates **Maybe Bool**
- ▶ **(Just 0)** has type **Maybe Int**, **(Just True)** has type **Maybe Bool**
- ▶ **Nothing** is **polymorphic**: it has type **Maybe a** for all possible *a*'s

We will get back to this in Week 3

We will see an example of this type in a minute

Datatypes, Pattern-Matching & Recursion

Lists of integers:

```
1 data IntList = Nil | Cons Int IntList
```

- ▶ A new type
- ▶ Nil is the empty list
- ▶ (Cons i l) is a list of composed of
 - ▶ a head: the integer i
 - ▶ a tail: the list l

Example:

```
1 somelist :: IntList  
2 somelist = Cons 2 (Cons 1 Nil)
```

Datatypes, Pattern-Matching & Recursion

Can we define a generic type of lists, not just for integers?

We can define a **parameterized List type constructor**

```
1 data List a = Nil | Cons a (List a)
```

Examples:

```
1 someintlist  :: List Integer
2 someintlist  = Cons 2 (Cons 1 Nil)
```

```
1 somefruitlist :: List Fruit
2 somefruitlist  = Cons Banana (Cons Apple Nil)
```

We cannot mix elements with different types: this is rejected:
Cons Apple (Cons 0 Nil)

Datatypes, Pattern-Matching & Recursion

```
1 data List a = Nil | Cons a (List a)
```

Write a function that returns the head of a list

```
1 head :: List a -> Maybe a
2 head Nil = Nothing
3 head (Cons h _) = Just h
```

- ▶ head **pattern-matches** on its argument
- ▶ a partial function: returns `Maybe`
- ▶ head is **polymorphic**: it can be applied to any kind of lists

```
1 somehead1 = head (Cons True (Cons False Nil)) -- returns (Just True)
2 somehead2 = head (Cons 0 (Cons 1 Nil)) -- returns (Just 0)
```


Datatypes, Pattern-Matching & Recursion

```
1 data List a = Nil | Cons a (List a)
```

A **recursive** function is a function that calls itself.

Example:

```
1 length :: (List a) -> Integer
2 length Nil = 0
3 length (Cons _ l) = 1 + length l
```

- ▶ length **pattern-matches** on its argument
- ▶ length is called inside the Cons branch
- ▶ length is **polymorphic**: it can be applied to any kind of lists

Higher-Order Functions

We can apply functions to functions:

```
1 map :: (a -> b) -> [a] -> [b]
2 map f [] = []
3 map f (x:xs) = (f x) : (map f xs)
4
5 somemap :: List Integer
6 somemap = map (\x -> 1+x) [0,1]
7 {-- returns [1,2] --}
```

- ▶ **map** takes a function and applies it to every element in a list.
- ▶ The type of **map** is actually $(a \rightarrow b) \rightarrow ([a] \rightarrow [b])$.
Only single-argument functions in Haskell!
- ▶ $(\backslash x \rightarrow x+1)$ is an anonymous function that increment numbers.
- ▶ **somemap** could also be written with named functions:

```
1 inc n = 1+n
2 somemap = map inc [0,1]
```

Partial Application

```
1 somemap :: List Integer
2 somemap = map (\x -> 1+x) [0,1]
3 {-- returns [1,2] --}
```

- ▶ In Haskell, $1+x$ is actually short for $(+) 1 x$.
- ▶ Try `ghci> :t (+)`

somemap could have been written as follows:

```
1 somemap = map ((+) 1) [0,1]
```

(Haskell lets you write $(1+)$ instead of $(+) 1$).

We have **partially applied** $(+)$.

Higher-Order Functions

We can even apply functions to functions:

```
1 map :: (a -> b) -> List a -> List b
2 map f Nil = Nil
3 map f (Cons h t) = Cons (f h) (map f t)
4
5 somemap :: List Integer
6 somemap = map (\x -> x+1) (Cons 0 (Cons 1 Nil))
7 {-- returns (Cons 1 (Cons 2 Nil)) --}
```

- ▶ `map` takes another function as argument
- ▶ and applies it to each member of the list
- ▶ `(\x -> x+1)` is an anonymous function that increment numbers
- ▶ it can be written as `(+1)`

Partial Application & Local Definitions

```
1 map :: (a -> b) -> List a -> List b
2 map f Nil = Nil
3 map f (Cons h t) = Cons (f h) (map f t)
4
5 somemap :: List Integer
6 somemap = map (\x -> x+1) (Cons 0 (Cons 1 Nil))
7 {-- returns (Cons 1 (Cons 2 Nil)) --}
```

somemap could have been written as follows:

```
1 add x y = x + y
2 somemap = map (add 1) (Cons 0 (Cons 1 Nil))
```

Note that (add 1) is **partially applied**.

It returns a function of type (Integer -> Integer)

Partial Application & Local Definitions

```
1 map :: (a -> b) -> List a -> List b
2 map f Nil = Nil
3 map f (Cons h t) = Cons (f h) (map f t)
4
5 somemap :: List Integer
6 somemap = map (\x -> x+1) (Cons 0 (Cons 1 Nil))
7 {-- returns (Cons 1 (Cons 2 Nil)) --}
```

somemap could also have been written as follows:

```
1 somemap =
2   let add x y = x + y in
3   map (add 1) (Cons 0 (Cons 1 Nil))
```

Where (add) is here a **local definition**,
which is only available inside **somemap**

Laziness

Haskell does not evaluate arguments until they are used:

```
1 replace :: b -> List a -> List b
2 replace f Nil = Nil
3 replace f (Cons h t) = Cons f (replace f t)
4
5 somereplace l =
6   let inc x = x + inc x in -- copy paste mistake!
7   replace (inc 1) l
```

- ▶ (somereplace Nil) returns Nil
- ▶ (somereplace (Cons 1 Nil)) runs for ever

Haskell is **Lazy**: does not evaluate arguments until they are needed

We will get back to that Week 2

Recursive definitions

```
1 longlist :: [Integer]
2 longlist = 1 : longlist
```

```
1 longlistaux = \ xs -> 1 : xs -- or (:) 1 or (1:)
2 longlist = longlistaux longlist
```

So `longlist` is a **fixed point** of `longlistaux`.

In Haskell, every endofunction has a fixed point, and we can construct fixed points uniformly:

```
1 fix :: (a -> a) -> a
2 fix f = f (fix f)
```

Could define `longlist = fix ((:) 1)`.

Conclusion

What did we cover today?

- ▶ Variables
- ▶ Types
- ▶ Functions
- ▶ Datatypes
- ▶ Pattern-matching
- ▶ Recursion
- ▶ Higher-Order Functions
- ▶ Laziness

Next time?

- ▶ The pure untyped λ -calculus